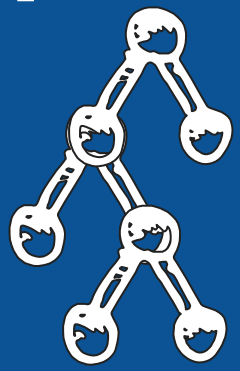
Алгоритмы и Алгоритмические Языки

Семинар #4:

1. Порядок вычисления в выражениях.
2. Память программы и указатели.
3. Передача возвращаемых значений из функций.
4. Одномерные массивы, циклы.

12.11.2025

Порядок вычисления в выражениях



Структура вычисления выражения

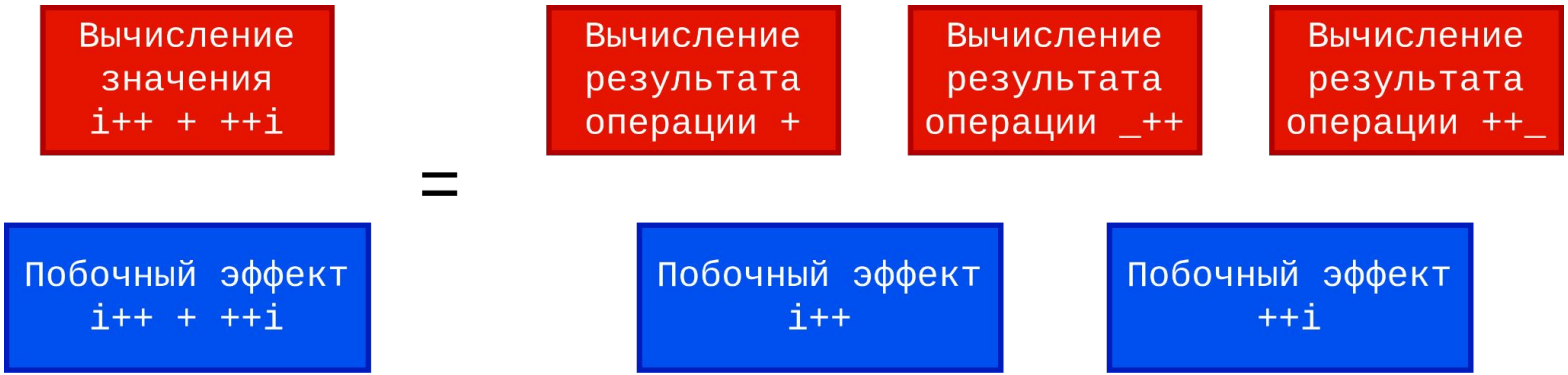
Каков будет результат вычисления набора операторов?

```
int i = 5;
int x = i++ + ++i;
printf(“%d\n”, x);
```

Структура вычисления выражения

Каков будет результат вычисления набора операторов?

```
int i = 5;
int x = i++ + ++i;
printf(“%d\n”, x);
```



Как взаимно упорядочены эти коробочки?

Отношение следования вычислений

Отношение **sequenced-before** (отношение “**следования до**”).

Опр. Для вычислений A и B верно $A \rightarrow B$, если и только если вычисление A заканчивается до начала вычисления B.

1. Если $A \rightarrow B$ и $B \rightarrow C$, то $A \rightarrow C$.
2. Если $A \rightarrow B$, то $B \not\rightarrow A$.
3. Если $A \not\rightarrow B$ и $B \not\rightarrow A$, то:
   - Либо A и B вычисляются в произвольном порядке.
   - Либо инструкции вычисления A и B перемешаны.
4. Для если между выражениями $\mathbb{A}$ и $\mathbb{B}$ находится точка следования, то для любых вычислений значений или побочных эффектов A и B верно $A \rightarrow B$.

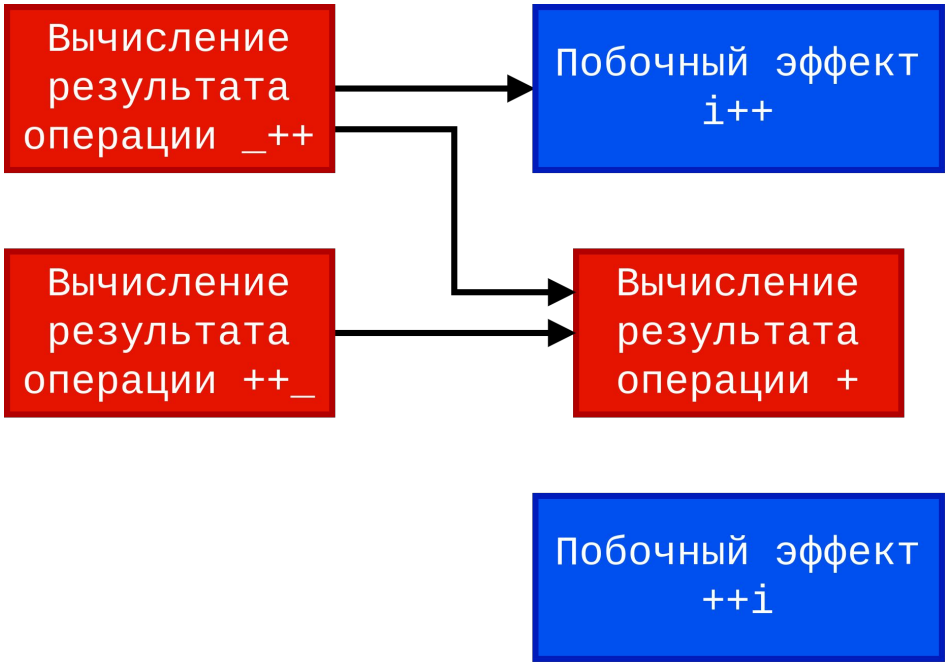
Но когда между $\mathbb{A}$ и $\mathbb{B}$ находится точка следования?

Точки следования

1. Некоторые операции задают точки следования для операндов:
   - В выражениях `A && B` и `A || B`, `A,B` будет `A → B`.
   - В выражении `A? B : C` будет `A → B` и `A → C`.
2. Вызов функции задаёт точку следования:
   В выражении `F(A, B)` верно:
   `A → <вызов ф-ии>`, `B → <вызов ф-ии>`, `F → <вызов ф-ии>`
3. Между операторами идёт точка следования:
   Для двух последовательных операторов: `A; B` будет `A → B`.
4. Точка следования находится после вычисления управляющего выражения в `if/for/while/do`.
5. Для операций `x++` и `x--` вычисление значения следует до побочного эффекта.
6. Для операции `A = B` будет `(вычисление B) → (запись в A)`.

Точки следования

Отношение следования для вычислений выражения `i++ + ++i`.



Undefined Behavior и следование

`Undefined Behavior` возникает тогда, когда:

1. Вычисления `A` и `B` (`A` $\nrightarrow$ `B` и `B` $\nrightarrow$ `A`) выполняют побочные эффекты над одной и той же переменной.

```
i = ++i + i++;      // undefined behavior
i = i++ + 1;        // undefined behavior
f(++i, ++i);        // undefined behavior
f(i = -1, i = -1); // undefined behavior
```

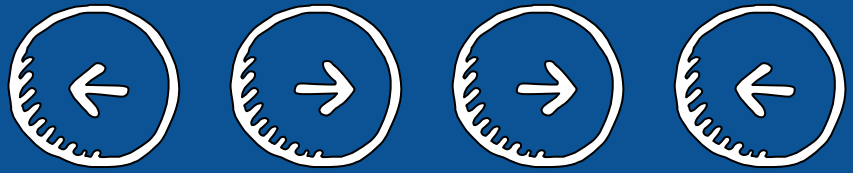
2. Вычисление `A` – это побочный эффект объекта, а `B` – использование объекта, `A` и `B` не упорядочены (`A` $\nrightarrow$ `B`, `B` $\nrightarrow$ `A`).

Порядок вычисления: задачи

Указать выражения, содержащие UB:

1. `i = i + 1`
2. `i = i++`
3. `i++ * i++`
4. `i++ && i++`
5. `a[i] = i++`
6. `a[i++] = i`
7. `i = f(i++)`
8. `i += i`
9. `a ^= b ^= a ^= b`
10. `a ^= b, b ^= a, a ^= b`
11. `i++ ? i++ : ++i`

Память программы и указатели



Адресация памяти компьютера

Память компьютера – адресуемый набор ячеек.

Размер одной ячейки – байт (часто – 8 бит).

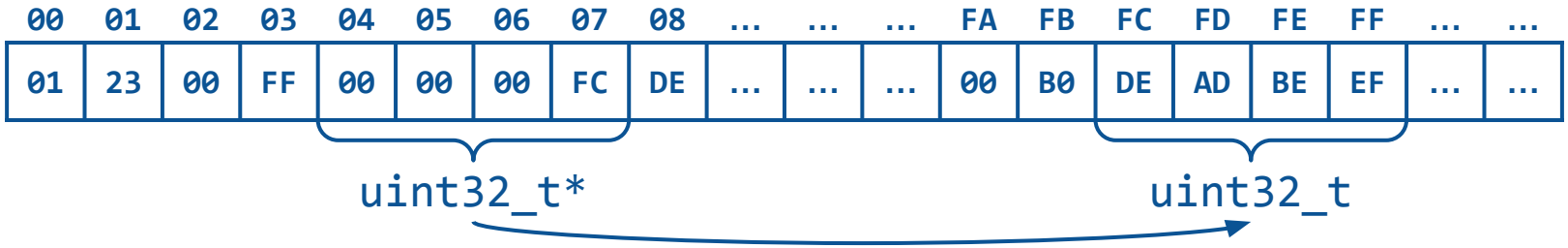
00	01	02	03	04	05	06	07	08	...	...	...	FA	FB	FC	FD	FE	FF	...	...
01	23	00	FF	DE	AD	BE	EF	DE	...	...	...	00	B0	1D	FA	CE	CA	...	...

В программе память выделена под:

- Исполняемый код вашей программы.
- Исполняемый код стандартной библиотеки.
- Локальные переменные.
- Глобальные переменные.
- Константные строки.

Указатели в языке Си

Тип данных, хранящий адрес ячейки памяти – **указатель**.
Обозначается добавление звёздочки к типу.




```
uint32_t  val = 0xDEADBEEF;
uint32_t* ptr = &val;
```

Взятие адреса переменной

```
*ptr = 0xB01DFACE;
*ptr += 0xBA1BEC;
```

Разыменование указателя

Операции с указателями: задачи

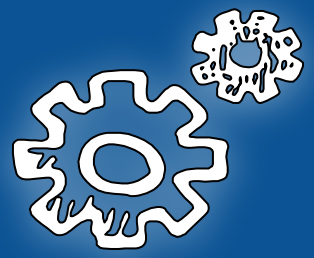
В программе объявлены следующие переменные:

```
int x; int* p; int** q;
```

Определить, какие из выражений являются корректными:

```
1) *p
2) *x
3) p = &x
4) &p
5) &(&x)
6) x = p
7) p = x
8) q = &x
9) q = &p
10) **p
11) *q
12) **q
13) ***q
14) &*p
```

Передача возвращаемых значений из функций



Применение указателей в функциях

См. пример **`04_square_equation`**:

- Ключевое слово **`const`** в указателях.
- Обращение по указателю **`NULL`**.
- Проверка указателей с помощью **`assert`**.

Одномерные массивы

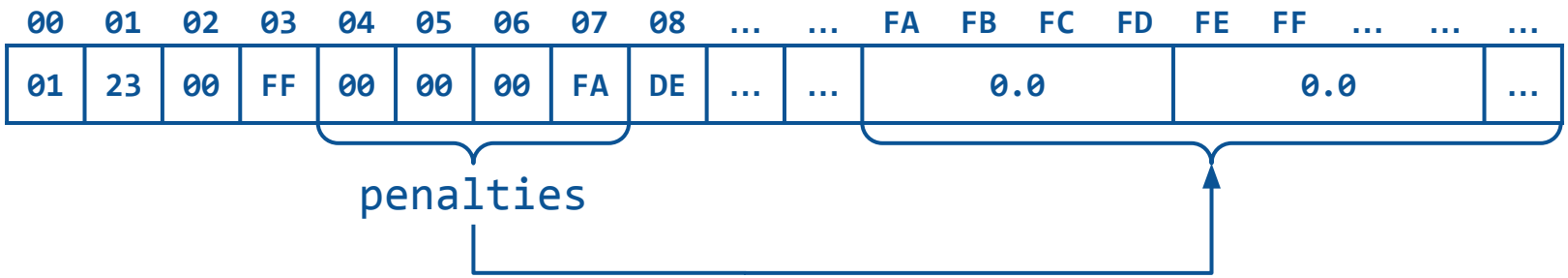


Массивы в языке Си

Объявление массива

```
float penalties[6] = {0.0, 0.0, 0.0, 0.0, 0.0, 0.0};
```

Нижележащее представление в памяти программы:



См. примеры: 01_grademe, 05_sum.

Передача массива в функцию

При передаче массива в функцию необходимо передавать длину:

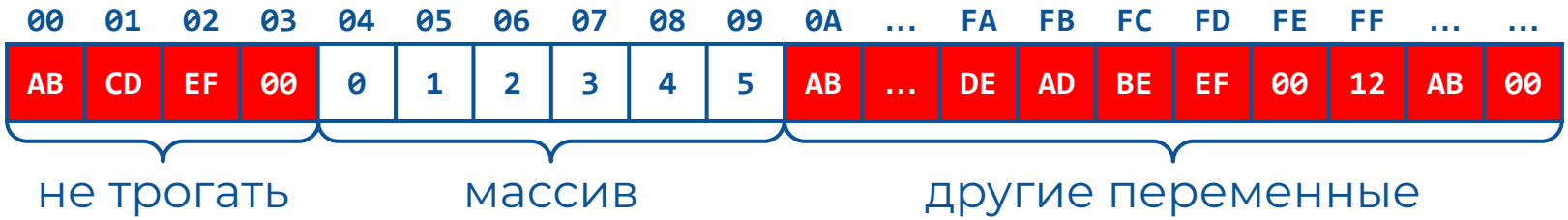
```
long sum(const int* array, unsigned length) {
    long sum = 0;

    for (int i = 0; i < length; i++) {
        sum += array[i];
    }

    return sum;
}
```

Выход за границы массива

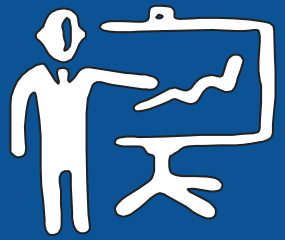
Доступ за границы массива – **Undefined Behavior (зло)**.



Возможные последствия выхода за границы массива:

- Портятся другие переменные.
- Программа “падает”.

Вопросы?



Красивые иконки взяты с сайта handdrawngoods.com